



RECAP

From last lecture...

46

**WHY ARGUMENT TO A COPY
CONSTRUCTOR MUST BE
PASSED AS A REFERENCE?**



ACTIVITY 01: COPY CONSTRUCTOR



```
#include <iostream>
using namespace std;

class Sample {
    int id;

public:
    void init(int x) { id = x; }
    void display() { cout << endl << "ID=" << id; }
};

int main()
{
    Sample obj1;
    obj1.init(10);
    obj1.display();

    Sample obj2(obj1);
    obj2.display();
    return 0;
}
```

49

ACTIVITY 02 : COPY CONSTRUCTOR

```
#include <iostream>
using namespace std;

class Sample {
    int *id=NULL;

public:
    void init(int x) {
        if (id == NULL) {
            id = new int; // Allocate memory
        }
        *id = x;
    }

    void display() {
        cout << endl << "ID = " << *id;
    }
};
```

```
int main() {
    int x = 100;

    Sample obj1;
    obj1.init(x);
    obj1.display();

    Sample obj2(obj1);
    obj2.display();

    int y = 200;
    obj2.init(y);
    obj1.display();
    obj2.display();

    return 0;
}
```



ACTIVITY



```

1  #include<iostream>
2  using namespace std;
3
4  class Hello{
5      string name;
6
7      public:
8
9      Hello();
10     Hello(string);
11
12     void display();
13
14     ~Hello(){
15         cout<<"Destructor is called: " <<name<<endl;}
16 };
17
18 Hello::Hello(){
19     name=" ";}
20
21 Hello::Hello(string n){
22     name= n;}
23
24 void Hello::display(){
25     cout<<"constructor is called and initialized the variable to: "<<name<<endl;}
26

```

```

27 int main()
28 {
29     Hello H1, H2("My Name");
30
31     Hello H3=H2;
32
33     H1.display();
34     H2.display();
35     H3.display();
36     H2.display();
37 }

```

```

constructor is called and initialized the variable to:
constructor is called and initialized the variable to: My Name
constructor is called and initialized the variable to: My Name
constructor is called and initialized the variable to: My Name
Destructor is called: My Name
Destructor is called: My Name
Destructor is called:

```




ACTIVITY



```

class Engine{
    private:

        int cylinders; // Number of cylinders
        double liters; // Capacity in liters

    public:

        Engine(int c = 4, double l = 1.3); // Constructor
};

Engine::Engine(int c, double l) : cylinders(c), liters(l)
{
    cout<<"Constructor Engine "<<endl;
    cout<<"Cylinders: "<<cylinders<<endl;
    cout<<"Liters: "<<litres<<endl;
}

```

```

class Car
{
    private:
        int modelNumber; // Model number
        Engine motor;     // Car's engine

    public:
        // Constructor
        Car(int m, int c, double l);
};

Car::Car(int m, int c, double l):modelNumber(m),
motor(c, l)
{
    cout<<"Constructor Car"<<endl;
    cout<<"Model No: "<<modelNumber<<endl;
}

int main(){
    Car c(2,3,2.3);
}

```

```

class Engine{
    private:

        ⑥ int cylinders; // Number of cylinders
        ⑦ double liters; // Capacity in liters

    public:

        Engine(int c = 4, double l = 1.3); // Constructor
};

⑤ Engine::Engine(int c, double l) : cylinders(c),
liters(l)
{ ⑧
    cout<<"Constructor Engine "<<endl;
    cout<<"Cylinders: "<<cylinders<<endl;
    cout<<"Liters: "<<litres<<endl;
} ⑨

```

```

class Car
{
    private:
        ④ int modelNumber; // Model number
        ⑤ Engine motor; // Car's engine

    public:
        // Constructor
        Car(int m, int c, double l);
};

③ Car::Car(int m, int c, double l):
modelNumber(m), motor(c, l)
{ ⑩
    cout<<"Constructor Car"<<endl;
    cout<<"Model No: "<<modelNumber<<endl;
}

① int main(){
    ② Car c(2,3,2.3);
}

```

57

DEFINING CLASS OBJECTS WITHIN A CLASS

Example: Creating a Cylinder

A SIMPLE CLASS

```
class Circle{  
    double radius;  
  
    public:  
        Circle(){ radius=0.0;}  
        double area();  
};
```

In OOP

- Class is the blueprint /template
- Use the template to create Objects
- Class includes class variables
 - Also known as data members / member variables
- Class Constructors
 - Executes / invoked when an instant is created using the class /create a new instance of a class.
- Member Functions
 - Get different functionalities

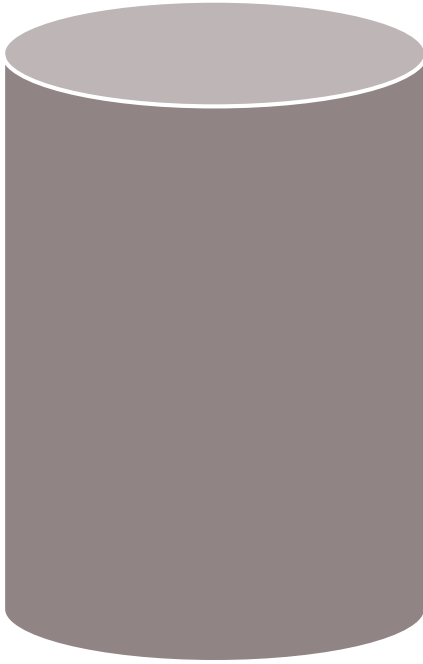
DEFINING CONSTRUCTORS

```
Circle ()  
{  
    radius = 0;  
}
```

```
Circle ():radius (0)  
{ }
```

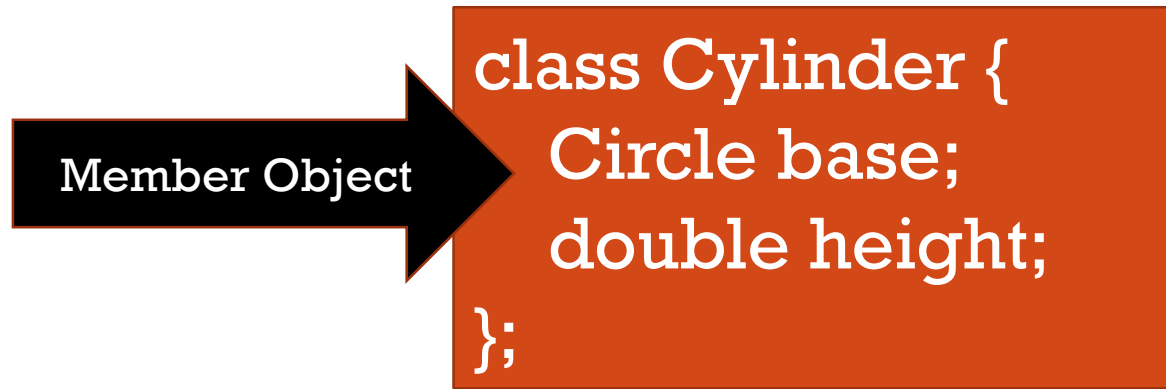
```
Circle (int r)  
{  
    radius = r;  
}
```

```
Circle (int r):radius (r)  
{ }
```

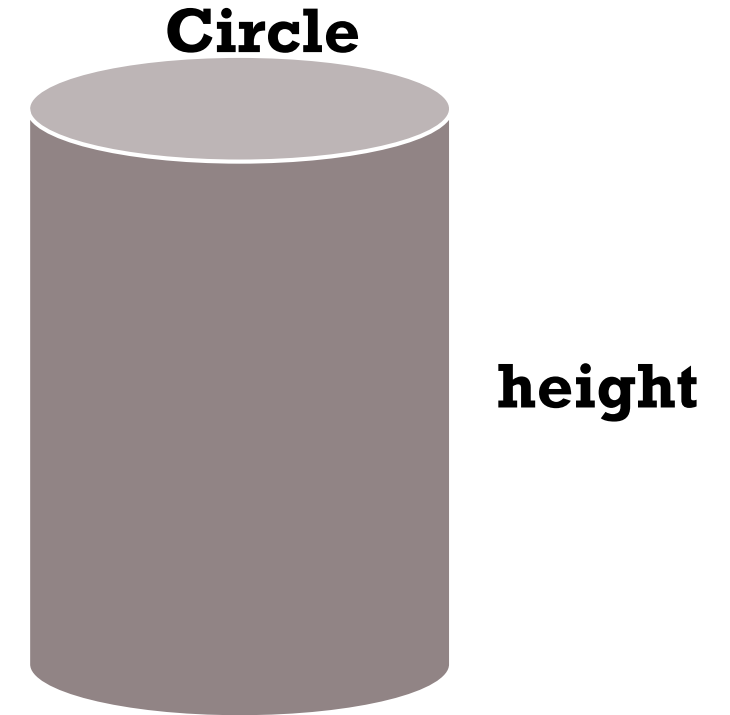


Cylinder

DEFINING CLASS OBJECTS WITHIN A CLASS



class Cylinder has a member object whose type is another class (In this example: Circle)



DEFINING CLASS OBJECTS WITHIN A CLASS

```
class Circle{
    double radius;
    public:
        Circle():radius(0) {
            cout<<"leaving the circle constructor" <<endl;
        }
        Circle (double r):radius(r){
            cout<<"leaving the Circle constructor: Radius
passed as a variable"<<endl;
        }
        double area();
};
```

```
class Cylinder {
    Circle base;
    double height;
    public:
        Cylinder(double r, double h) : height(h), base(r)
        {
            cout<<"leaving the Cylinder constructor"<<endl;
        }
        double volume() {return base.area() * height;}
};
```

Cylinder Cyl1(3,4);

DEFINING CLASS OBJECTS WITHIN A CLASS

1. Create a class object “Cyl1” of the type “Cylinder”.
2. The constructor will be called.
`Cylinder(double h, double r) : height(h), base(r)`
3. Create a double variable named height and assign it a value of h.
4. Create a class object name “base” and call the corresponding constructor to initialize the variables of object “base”.
5. The Circle constructor will be called.
`Circle (double r):radius(r){
cout<<"leaving the Circle constructor: Radius passed as a variable"<<endl; }`
6. End of the Circle constructor.
7. End of the Cylinder constructor.
8. End of the main() function.

`Cylinder Cyl1(3,4);`

STATIC KEYWORD IN C++

- A static variable is initialized only once and stored in the static storage area.
- It retains its value between function calls and exists until the program terminates.
- Static Keyword can be used to define,
 - Local variables,
 - Class member variables,
 - Functions.

STATIC LOCAL VARIABLES

- The variables declared as static are initialized only once.
- They are allocated space in separate static storage.
- Space for **it gets allocated for the lifetime of the program.**
- The value of variable in the previous call gets carried through the next function call.
- Even if the function is called multiple times, space for the static variable is allocated only once

EXAMPLE

```
0 0
1 0
2 0
3 0
4 0

-----
Process exited after 0.1813 seconds with return value 0
Press any key to continue . . .
```



```
void demo()
{
    static int count = 0;
    int c = 0;

    cout << count << " " << c << endl;

    count++;
    c++;
}

int main()
{
    for (int i=0; i<5; i++)
        demo();
}
```

STATIC MEMBER VARIABLES IN A CLASS

- The static variables **in a class are shared by the objects.**
- Can not have multiple copies of the same static variables for different objects.
- Can not be initialized using constructors, as these are not dependent on object initialization
- A static variable inside a class should be initialized explicitly, always outside the class using the class name and scope resolution operator outside the class.

EXAMPLE:

Type ClassName::variable_name = value;

ACCESSING STATIC VARIABLES IN A CLASS

- A member function can refer to a static data member of the same class directly.
- A non-member function can refer to a static data member using either the notation

`ClassName::staticMemberName` or
`objectName.staticMemberName`

STATIC CLASS OBJECTS

- Like variables, objects also when declared as static have a scope till the lifetime of the program.
- Exists the scope at the end of the program.

EXERCISE

```
int main()
{
    int x=0;
    if(x==0){
        static StaticObject obj;
    }
    cout << "End of main\n";
}
```

```
class StaticObject
{
    int i;
public:

    StaticObject():i(0)
    {
        cout << "Inside Constructor"<<endl;
    }

    ~StaticObject()
    {
        cout << "Inside Destructor\n";
    }
};
```

STATIC MEMBER FUNCTIONS

- Work for the class as a whole rather than for a particular object of a class.
 - Does not depend on the object of a class.
- It can be called using an object and the direct member access “.” operator.
 - Recommended invoking the static members using the class name and the scope resolution operator.
- Allowed to access only the static data members or other static member functions
 - Can not access the non-static data members or member functions of the class.

STATIC MEMBER FUNCTIONS

- Like static data members, static member functions are associated with a class and not with any particular object of that class.
- Static member functions do not have a **this** pointer.
- Static member functions cannot be declared **const**.
- A member function can call a static member function of the same class directly.
- A non-member function can call a static member function using either the below notation

`ClassName::staticMemberName()` or
`objectName.staticMemberName()`

```

1  #include <iostream>
2  using namespace std;
3
4  class Counter {
5      static int count;
6
7  public:
8      static void increment() {
9          count++;
10     }
11
12     static void display() {
13         cout << "Current count is: " << count << endl;
14     }
15 };

```

```

17 int Counter::count = 0;
18
19 int main() {
20     Counter::display();
21     Counter::increment();
22     Counter::increment();
23     Counter::display();
24
25     Counter c1, c2;
26     c1.increment();
27     Counter::display();
28
29     return 0;
30 }

```

```

Current count is: 0
Current count is: 2
Current count is: 3

```

```

-----
Process exited after 1.031 seconds with return value 0
Press any key to continue . . .

```

CONSTANTS

- Variables
- Pointers
- Function arguments and return types
- Class Data members
- Class Member functions
- Objects

CONSTANT VARIABLES IN C++

- Cannot change its value of the variable.
- Value of the variable will not change during its lifetime.
- Constant variables must be initialized while they are declared.

```
int main()
{
    const int i = 10;
    const int j = i + 10;
    i++;
}
```



Compilation error

POINTERS WITH CONST KEYWORD

Declaration can be done in THREE ways.

- Pointer to a constant variable (`const int* ptr`)
 - The pointer is pointing to a constant value.
 - Cannot change the value through the pointer.
 - Can change the pointer to point to another variable.
- Constant Pointer (`int* const ptr`)
 - The pointer is a constant pointer.
 - Cannot change the pointer, i.e., it will always point to the same memory location.
 - Can change the value at the memory location it points to.
- Constant Pointer to Constant Variable (`const int* const ptr`)
 - The pointer is constant and it points to a constant value.
 - Cannot change the pointer.
 - Cannot change the value through the pointer.

```
const int* u;
```

```
int x = 1;  
int* const w = &x;
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    int a = 10;
    int b = 20;
    const int c = 30;
```

```
    const int* ptrToConst = &a;
    cout << "ptrToConst points to a = " << *ptrToConst << endl;
```

```
    ptrToConst = &c;
    cout << "ptrToConst now points to c = " << *ptrToConst << endl;
```

```
    int* const constPtr = &a;
    *constPtr = 25;
    cout << "constPtr points to a with new value = " << *constPtr << endl;
```

```
    const int* const constPtrToConst = &c;
    cout << "constPtrToConst points to c = " << *constPtrToConst << endl;
    return 0;
}
```

Compile?

CONSTANT FUNCTION ARGUMENTS AND RETURN TYPES

- The return type or arguments of a function can be defined as constant.
- Member functions should not modify member data.

```
void func(const int i)
{
    i++; // error
}

const int funcCont()
{
    return 6;
}
```

DEFINING CLASS DATA MEMBERS AS CONSTANT

- Data members are not initialized during declaration.
- The initialization is done in the constructor.
- Once initialized cannot change the value.

Exercise:

Write a function called setValue within the class "TestConst" and try to change the variable 'var'.

Compiles correctly

```
class TestConst
{
    const int var;
public:
    TestConst(int x):var(x)
    {
        cout << "Const Var: " <<
var<<endl;
    }
};

int main()
{
    TestConst testObj1(3);
    TestConst testObj2(2);
}
```

DEFINING CLASS OBJECT AS CONSTANTS

- During the object lifetime Data members can never be changed.

```
const class_name object;
```

CONSTANT OBJECTS

- Objects are seldom passed by value in C++.
- If you want to pass objects, they are passed by passing a **const** reference
- Passing by **const** reference
 - Avoids copying the object
 - Protects the object from unintentional changes.

```
void displayTime(const Time &t)
```

DEFINING CLASS MEMBER FUNCTION AS CONSTANTS

- **const keyword** must be added to both the function signature and implementation.
- A constant member functions never modifies data members in an object.

```
returnType functionName() const
```

```
int displayList() const;
```

```
int List::displayList () const  
{  
    .....  
}
```

83

'THIS' KEYWORD

It points to the **object** that is currently invoking the method

Rectangle.cpp

THE “THIS” POINTER IN C++

- How do objects work with functions and data members of a class?
- When an object is created
 - Each object gets its own copy of the data member.
 - All-access the same function definition as present in the code segment.
- Each object gets its own copy of data members and share a single copy of member functions.
- How the proper data members are accessed and updated?
 - An implicit pointer ‘this’ (value is the address of the object that generated the call)
 - Passed as a hidden argument to all non-static member function calls and
 - Available as a local variable within the body of all non-static functions.
 - ‘this’ pointer is not available in static member functions as static member functions can be called without any object (with class name).

IN RECTANGLE EXAMPLE...

- **this** points to the object that invoked **getWL()**.

```
void Rectangle::getWL()
{
    cout<<"Width is "<<this->width<<endl;
    cout<<"Length is "<<this->length<<endl;
}
```